SI2

2.ENTREGA

ERREFAKTORIZAZIOA

Egileak:

- Enara Anitua Goitiandia
- Hode Salegi Aranguena
- Jon Elortegi Mendizabal

Aurkibidea

1. SONARCLOUD-eko hasierako egoera: 3

2. "WRITE SHORT UNITS OF CODE" (2. KAPITULUA) 3

Jon Elortegi Mendizabal 3

Enara Anitua Goitiandia 4

Hodei Salegi Aranguena 5

3. "WRITE SHORT UNITS OF CODE" (3.KAPITULUA) 6

Jon Elortegi Mendizabal 6

Enara Anitua Goitiandia 7

Hodei Salegi Aranguena 8

4. "DUPLICATE COED" (4.kapituloa) 10

Jon Elortegi Mendizabal 10

Enara Anitua Goitiandia 12

Hodei Salegi Aranguena 17

5. "KEEP UNIT INTERFACES SMALL" (5.Kapitulua) 21

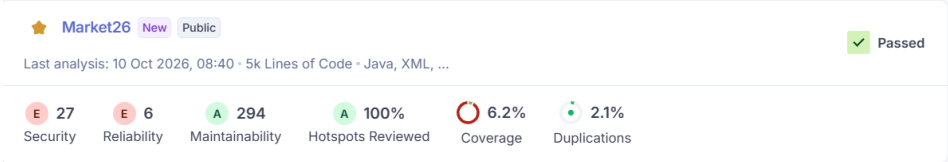
Jon Elortegi Mendizabal 21

Enara Anitua Goitiandia 22

Hodei Salegi Aranguena 24

6. ONDORIOAK 26

1. SONARCLOUD-eko hasierako egoera:



2. “WRITE SHORT UNITS OF CODE” (2. KAPITULUA)

Jon Elortegi Mendizabal

Aukeratutako Metodoa:

```
public boolean createReclamation(int saleNumber, String deskribapena, String buyerEmail) {
    if (deskribapena == null || buyerEmail == null) return false;
    db.getTransaction().begin();
    try {
        // 1. Bilatu datu-basean beharrezko objektuak
        Sale s = db.find(Sale.class, saleNumber);
        Seller eroslea = db.find(Seller.class, buyerEmail);

        if (s != null && eroslea != null) {
            Seller saltzailea = s.getSeller();

            // 2. Erreklamazioa sortu (4 parametro: deskribapena, produktua, saltzailea, eroslea)
            domain.Reclamation r = new domain.Reclamation(deskribapena, s, saltzailea, eroslea);

            // 3. Loturak egin objektuen artean
            s.setReclamation(r); // Produktuak badaki erreklamazio bat duela
            saltzailea.addErreklamazioa(r); // Saltzaileak badaki erreklamazio bat jaso duela

            // 4. Gorde erreklamazioa datu-basean
            db.persist(r);

            db.getTransaction().commit();
            db.refresh(saltzailea);
            return true;
        }

        if (db.getTransaction().isActive()) db.getTransaction().rollback();
        return false;

    } catch (Exception e) {
        if (db.getTransaction().isActive()) db.getTransaction().rollback();
        e.printStackTrace();
        return false;
    }
}
```

Kode ondoren:

```
public boolean createReclamation(int saleNumber, String deskribapena, String buyerEmail) {
    if (deskribapena == null || buyerEmail == null) return false;
    db.getTransaction().begin();
    try {
        return exekutatuErreklamazioa(saleNumber, deskribapena, buyerEmail);
    } catch (Exception e) {
        if (db.getTransaction().isActive()) db.getTransaction().rollback();
        e.printStackTrace();
        return false;
    }
}
private boolean exekutatuErreklamazioa(int saleNumber, String deskribapena, String buyerEmail) {
    // 1. Bilatu datu-basean beharrezko objektuak
    Sale s = db.find(Sale.class, saleNumber);
    Seller eroslea = db.find(Seller.class, buyerEmail);
    if (s != null && eroslea != null) {
        Seller saltzailea = sortuEtaLotuErreklamazioa(deskribapena, s, eroslea);
        db.getTransaction().commit();
        db.refresh(saltzailea);
        return true;
    }
    if (db.getTransaction().isActive()) db.getTransaction().rollback();
    return false;
}
private Seller sortuEtaLotuErreklamazioa(String deskribapena, Sale s, Seller eroslea) {
    Seller saltzailea = s.getSeller();
    // 2. Erreklamazioa sortu (4 parametro: deskribapena, produktua, saltzailea, eroslea)
    domain.Reclamation r = new domain.Reclamation(deskribapena, s, saltzailea, eroslea);
    // 3. Loturak egin objektuen artean
    s.setReclamation(r); // Produktuak badaki erreklamazio bat duela
    saltzailea.addErreklamazioa(r); // Saltzaileak badaki erreklamazio bat jaso duela
    // 4. Gorde erreklamazioa datu-basean
    db.persist(r);
    return saltzailea;
}
```

Aukeratutako metodoa: DataAccess klaseko createReclamation metodoa.

Zergatik aukeratu den: Jatorrizko metodo honek 30 lerro baino gehiago zituen, eta "Building Maintainable Software" liburuko bigarren kapituluak ezartzen duen irizpidea urratzen zuen. Zehazki, *"Write short units of code"* gomendioak unitateen/metodoen luzera **gehienez 15 lerrokoa** izatea mugatzen du, mantentze-lanak errazteko.

Zer egin den: Eclipse inguruneko **Extract Method** errefaktorizazio-tresna erabiliz, metodoaren barruko logika nagusia (erreklamazioaren sorrera eta objektuen arteko loturak kudeatzen dituen zatia) metodo laguntzaile berri batera atera da (exekutatuErreklamazioa). Hala ere, sortutako metodo laguntzaileak, 15 lerro baino gehiago zituen, beraz berrez errefaktorizatu dut beste metodo bat sortuz. Horrela, metodo nagusiaren eta metodo laguntzaileen lerro-kopurua 15etik behera murriztu da, jatorrizko funtzionalitatea eta portaera zehatz-mehatz mantenduz.

Enara Anitua Goitiandia:

Aukeratutako Metodoa:

```
public boolean removeSale(String email, Sale sale) {
    // 3. puntua: Balidazioa (null-ak ekiditeko)
    if (email == null || sale == null) return false;

    db.getTransaction().begin();
    try {
        Seller seller = db.find(Seller.class, email);
        Sale s = db.find(Sale.class, sale.getSaleNumber());

        if (seller != null && s != null && s.getSeller().getEmail().equals(email)) {

            // 1. puntua: Erosita badago, ezin da ezabatu
            if (s.isSold()) {
                System.out.println("ERROREA: Ezin da ezabatu, produktua jada salduta dago.");
                db.getTransaction().rollback();
                return false;
            }

            seller.getSales().remove(s);
            db.remove(s);
            db.getTransaction().commit();
            return true;
        }
        return false;
    } catch (Exception e) {
        if (db.getTransaction().isActive()) db.getTransaction().rollback();
        return false;
    }
}
```

Kode ondoren:

```
public boolean removeSale(String email, Sale sale) {
    // 3. puntua: Balidazioa (null-ak ekiditeko)
    if (email == null || sale == null) return false;

    db.getTransaction().begin();
    try {
        Seller seller = db.find(Seller.class, email);
        Sale s = db.find(Sale.class, sale.getSaleNumber());

        if (seller != null && s != null && s.getSeller().getEmail().equals(email)) {

            // 1. puntua: Erosita badago, ezin da ezabatu
            return salmentaEzabapena(seller, s);
        }
        return false;
    } catch (Exception e) {
        if (db.getTransaction().isActive()) db.getTransaction().rollback();
        return false;
    }
}

private boolean salmentaEzabapena(Seller seller, Sale s) {
    if (s.isSold()) {
        System.out.println("ERROREA: Ezin da ezabatu, produktua jada salduta dago.");
        db.getTransaction().rollback();
        return false;
    }

    seller.getSales().remove(s);
    db.remove(s);
    db.getTransaction().commit();
    return true;
}
```

Aukeratutako metodoa**:** DataAccess klaseko removeSale metodoa.

Zergatik aukeratu den: Jatorrizko removeSale metodo honek 15 lerro baino gehiago zituen (18 lerro zehazki). Baina, “Buildin Maintanable Software” liburuko bigarren kapituluak ezartzen duen irizpidea ez du betetzen kode honek. Bigarren kapituluak zera dio: "Write short units of code" euskaraz, idatzi kode-unitate motzak, hau da, gomendatzen du kodea laburra izan behar dela. Zehazki, gehienez 15 lerro izatea eskatzen du mantentze-lanak errazteko.

Zer egin den: Lehenik, ondoren metodo bihurtuko dugu kode zati bat aukeratuko dugu. Behin kode zatia hautatzean eskuineko botoiari sakatu eta ondorengoa egingo dugu: Refactor -> Extract Method. Atal horrek ondorengoa egiten du: removeSale metodoaren barruko logika nagusia (salmentaren ezabapena eta balidazioak kudeatzen dituen zatia) metodo laguntzaile berri batera atera da (salmentaEzabapena). Horrela, metodo nagusiaren lerro-kopurua 15etik behera murriztu da (12 lerro), funtzionalitatea eta portaera mantenduz.

Hodei Salegi Aranguena

Aukeratutako Metodoa:

```
public Sale createOffer(CreateOfferParams params, Demand demand, Seller seller) {
    db.getTransaction().begin();
    try {
        Demand d = db.find(Demand.class, demand.getDemandNumber());
        Seller s = db.find(Seller.class, seller.getEmail());

        Sale offer = new Sale();
        offer.setTitle(params.getTitle());
        offer.setPrice(params.getPrice());
        offer.setDescription(params.getDescription());
        offer.setSeller(s);
        offer.setPublicationDate(new java.util.Date());
        offer.setParentDemand(d);
        return offer;

        db.persist(offer);
        s.addSale(offer);
        db.getTransaction().commit();
        return offer;
    } catch (Exception e) {
        if (db.getTransaction().isActive()) db.getTransaction().rollback();
        e.printStackTrace();
        return null;
    }
}
```

Kode ondoren:

```java
public Sale createOffer(CreateOfferParams params, Demand demand, Seller seller) {
    db.getTransaction().begin();
    try {
        Demand d = db.find(Demand.class, demand.getDemandNumber());
        Seller s = db.find(Seller.class, seller.getEmail());

        Sale offer = eskaintzaOsatu(params, d, s);

        db.persist(offer);
        s.addSale(offer);
        db.getTransaction().commit();
        return offer;
    } catch (Exception e) {
        if (db.getTransaction().isActive()) db.getTransaction().rollback();
        e.printStackTrace();
        return null;
    }
}
```

Aukeratutako metodoa: DataAccess klaseko createOffer metodoa.

Zergatik aukeratu den: Jatorrizko createOffer metodo honek 15 lerro baino gehiago zituen (25 lerro zehazki). Baina, "Building Maintainable Software" liburuko bigarren kapituluak ezartzen duen irizpidea ez du betetzen kode honek. Bigarren kapituluak zera dio: "Write short units of code" (euskaraz, idatzi kode-unitate laburrak), hau da, kode-unitate bakoitzaren luzera muga dezala eskatzen du mantentze-lanak eta berrerabilgarritasuna errazteko. Zehazki, gehienez 15 lerro izatea gomendatzen du.

Zer egin den: Lehenik, ondoren azpi-metodo bihurtuko dugu kode zatia aukeratuko dugu (Sale objektua sortu eta bere atributuak ezartzen dituzten lineak). Behin kode zati hori hautatuta, eskuineko botoiari sakatu eta ondorengoa egingo dugu Eclipse-n: Refactor -> Extract Method. Refaktorizazio honek ondorengoa egiten du: createOffer metodoaren barruko Sale objektuaren eraikuntza- eta konfigurazio-logika metodo laguntzaile berri batera atera du (eraikiEskaintza). Horrela, metodo nagusiaren lerro-kopurua 15etik behera murriztu da (14 lerro) eta sortutako azpi-metodoak 9 lerro bakarrik ditu, jatorrizko funtzionalitatea eta portaera erabat mantenduz.

3. “WRITE SHORT UNITS OF CODE” (3.KAPITULUA)

Jon Elortegi Mendizabal

Hasierako Kodea:

```java
/**
 * Produktu bat erabiltzailearen saskian gehitzen du.
 */
public boolean addToBasket(String buyerEmail, int saleNumber) {
    db.getTransaction().begin();
    try {
        Seller buyer = db.find(Seller.class, buyerEmail);
        Sale s = db.find(Sale.class, saleNumber);

        if (buyer != null && s != null && !s.isSold()) {
            buyer.addBasket(s); // Orain Seller-ek mugarik gabe gehitzen du
            db.getTransaction().commit();
            return true;
        }
        if (db.getTransaction().isActive()) db.getTransaction().rollback();
        return false;
    } catch (Exception e) {
        if (db.getTransaction().isActive()) db.getTransaction().rollback();
        return false;
    }
}
```

Kodea Ondoren:

```
/**
 * Produktu bat erabiltzailearen saskian gehitzen du.
 */
public boolean addToBasket(String buyerEmail, int saleNumber) {
    db.getTransaction().begin();
    try {
        Seller buyer = db.find(Seller.class, buyerEmail);
        Sale s = db.find(Sale.class, saleNumber);

        if (gehituSaskiraLogika(buyer, s)) {
            return true;
        }

        if (db.getTransaction().isActive()) db.getTransaction().rollback();
        return false;
    } catch (Exception e) {
        if (db.getTransaction().isActive()) db.getTransaction().rollback();
        return false;
    }
}

private boolean gehituSaskiraLogika(Seller buyer, Sale s) {
    if (buyer != null && s != null && !s.isSold()) {
        buyer.addBasket(s);
        db.getTransaction().commit();
        return true;
    }
    return false;
}
```

Aukeratutako metodoa: DataAccess klaseko addToBasket metodoa.

Zergatik aukeratu den: "Building Maintainable Software" liburuko hirugarren kapituluak ezartzen duen irizpideari jarraituz, unitate bakoitzeko adar-puntu kopurua (branch points) gehienez 4ra mugatu behar da. Jatorrizko addToBasket metodoak baldintza konplexuak, transakzioen egiaztapenak eta salbuespen-kudeaketak zituenez, adar-puntu gehiegi zituen (gomendatutako muga gaindituz).

Zer egin den: Metodo nagusia zatitu egin dut, metodo laguntzaile bat sortuz. Horrela, metodo nagusiaren zein laguntzailearen adar-puntuak zatitu eta murriztu egin dira, kodea askoz ere sinpleago, testatzen errazagoa eta mantentzen lagungarriagoa bihurtuz.

Enara Anitua Goitiandia:

Hasierako Kodea:

```
public Seller register(String email, String name, String password) {

    if (email == null || name == null || password == null) return null;

    db.getTransaction().begin();
    Seller s = db.find(Seller.class, email);
    if (s != null) {
        db.getTransaction().commit();
        return null;
    }
    Seller newSeller = new Seller(email, name, password);
    db.persist(newSeller);
    db.getTransaction().commit();
    return newSeller;
}
```

Kodea ondoren:

```java
public Seller register(String email, String name, String password) {

    if (NullDu(email, name, password)) return null;

    db.getTransaction().begin();
    Seller s = db.find(Seller.class, email);
    if (s != null) {
        db.getTransaction().commit();
        return null;
    }
    Seller newSeller = new Seller(email, name, password);
    db.persist(newSeller);
    db.getTransaction().commit();
    return newSeller;
}

private boolean NullDu(String email, String name, String password) {
    return email == null || name == null || password == null;
}
```

Aukeratutako metodoa: DataAccess klaseko register metodoa.

Zergatik aukeratu den: “Building Maintanable Software” liburuko 3.kapituluak ezartzen duen irizpideak dio gehienez 4 adar-puntu izan ditzakeela gehinez metodoak (if, try-catch…). Jatorrizko register metodoak parametroen nulutasuna egiaztatzeko baldintza konplexua eta datu-basean erabiltzailea dagoen ala ez egiaztatzeko beste baldintza bat zituenez, muga-mugarrean zegoen.

Zer egin den: Metodo nagusia zatitu egin dut (email == null || name == null || password == null), eta horrekin metodo laguntzaile bat sortu dut (NullDa). Refactor -> Extract Method eginez. Horrela, metodo nagusian 4 adar-puntu baino gutxiago daude eta berdina metodo laguntzailean. Beraz, kodea askoz sinpleagoa da eta frogatzerako eta mantentze orduan errazagoa izango da.

Hodei Salegi Aranguena

Hasierako Kodea:

```java
public boolean kontuaItxi(String email) {
    if (email == null) return false;

    db.getTransaction().begin();
    try {
        Seller s = db.find(Seller.class, email);

        if (s != null) {
            // 2. Eskaerak (Demands) ezabatu
            try {
                TypedQuery<Demand> queryD = db.createQuery("SELECT d FROM Demand d WHERE d.claimant.email = :email", Demand.class);
                queryD.setParameter("email", email);
                List<Demand> demands = queryD.getResultList();
                for (Demand d : demands) {
                    db.remove(d);
                }
            } catch (Exception e) {
                System.out.println("Oharra: Ez da eskaerarik aurkitu");
            }

            // 3. Produktuak (Sale) desgaitu BAINA EZ EZABATU
            // Bakarrik desgaitu erosleen historian datuak gera daitezen
            try {
                for (Sale sale : s.getSales()) {
                    sale.setAvailable(false); // Ez da dendan agertuko, baina DBan jarraituko du
                }
            } catch (Exception e) {
                System.out.println("Oharra: Errorea produktuak desgaitzean");
            }

            // 4. Erabiltzailea "desgaitu"
            s.setAktibo(false);

            // 5. Zerrenda beltzera gehitu
            BlacklistedUser bu = new BlacklistedUser(email);
            db.persist(bu);

            db.getTransaction().commit();
            return true;
        } else {
            if (db.getTransaction().isActive()) db.getTransaction().rollback();
            return false;
        }
    } catch (Exception e) {
        if (db.getTransaction().isActive()) db.getTransaction().rollback();
        e.printStackTrace();
        return false;
    }
}
```

Kodea ondoren:



```
public boolean kontualtxi(String email) {
    if (email == null) return false;

    db.getTransaction().begin();
    try {
        Seller s = db.find(Seller.class, email);

        if (s != null) {
            // 2. Eskaerak (Demands) ezabatu
            ezabatuEskaerak(email);

            // 3. Produktuak (Sale) desgaitu BAINA EZ EZABATU
            // Bakarrik desgaitu erreferentzia historian datuak mesa daitezen
            desgaituSalmentak(s);

            // 4. Erabiltzailea "desgaitu"
            s.setAktibo(false);

            // 5. Zerrenda beltzera gehitu
            BlacklistedUser bu = new BlacklistedUser(email);
            db.persist(bu);

            db.getTransaction().commit();
            return true;
        } else {
            if (db.getTransaction().isActive()) db.getTransaction().rollback();
            return false;
        }
    } catch (Exception e) {
        if (db.getTransaction().isActive()) db.getTransaction().rollback();
        e.printStackTrace();
        return false;
    }
}

private void desgaituSalmentak(Seller s) {
    try {
        for (Sale sale : s.getSales()) {
            sale.setAvailable(false); // Ez da ezabatu egiten, baizik DBan desgaituta da
        }
    } catch (Exception e) {
        System.out.println("Oharra: Errorea produktuak desgaitzean");
    }
}

private void ezabatuEskaerak(String email) {
    try {
        TypedQuery<Demand> query0 = db.createQuery("SELECT d FROM Demand d WHERE d.claimant.email = :email", Demand.class);
        query0.setParameter("email", email);
        List<Demand> demands = query0.getResultList();
        for (Demand d : demands) {
            db.remove(d);
        }
    } catch (Exception e) {
        System.out.println("Oharra: Ez da eskaerarik aurkitu");
    }
}
```

Aukeratutako metodoa: DataAccess klaseko kontualtxi metodoa.

Zergatik aukeratu den: Jatorrizko kontualtxi metodoak adarkatze-puntu gehiegi zituen (6 adarkatze-puntu, if habiaratuen, habiaratutako try-catch anitzen eta bigizten ondorioz). "Building Maintainable Software" liburuko hirugarren kapituluak ezartzen duen irizpidea ez du betetzen kode honek. Hirugarren kapituluak zera dio: "Write simple units of code" (euskaraz, idatzi kode-unitate sinpleak), hau da, unitate bakoitzeko gehienez 4 adarkatze-puntu izatea gomendatzen du kodearen konplexutasuna txikitzeko eta probak nahiz mantentze-lanak errazteko.

Zer egin den: Metodoaren konplexutasuna murrizteko, barneko try-catch bloke independenteak azpi-metodo sinpleetan banatu dira Eclipse-ko Refactor -> Extract Method tresna erabiliz. Hain zuzen ere, erabiltzailearen eskaerak ezabatzea ezabatuEskaerak azpi-metodora atera da, eta produktuen desgaitzea desgaituSalmentak azpi-metodoan isolatu da. Horrela, kontualtxi metodo nagusiaren adarkatze-kopurua 3ra murriztu da (gehienezko 4ko mugaren azpitik utziz), kodearen irakurgarritasuna hobetuz eta mantentzea askoz errazagoa eginez.

4. “DUPLICATE CODE” (4.kapituloa)

Jon Elortegi Mendizabal

Hasierako Kodea:

```
package gui;

import java.awt.*;

public class BizumGUI extends JFrame {
    private float kopurua;
    private final Seller user;
    private JFrame previousFrame;
    private boolean isSarrera;

    public Seller getUser() {
        return user;
    }

    public BizumGUI(float kopurua, Seller user, JFrame previousFrame, boolean isSarrera) {
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        ImageIcon icon = new ImageIcon(getClass().getClassLoader().getResource("images/Logo.png"));
        setIconImage(icon.getImage());
        this.kopurua = kopurua;
        this.user = user;
        this.previousFrame = previousFrame;
        this.isSarrera = isSarrera;

        setTitle(ResourceBundle.getBundle("Etiquetas").getString("BizumGUI.Title"));
        setSize(400, 300); // Pixka bat handitu dugu botoi berria sartzeko
        getContentPane().setLayout(null);
        this.getContentPane().setBackground(Color.WHITE);

        JLabel lblInfo = new JLabel((isSarrera ? ResourceBundle.getBundle("Etiquetas").getString("BizumGUI.Add")+": " : ResourceBundle.getBundle("Etiquetas").getSt
        lblInfo.setBounds(50, 20, 300, 30);
        lblInfo.setHorizontalAlignment(SwingConstants.CENTER);
        lblInfo.setFont(new Font("Segoe UI", Font.BOLD, 16));
        getContentPane().add(lblInfo);

        JLabel lblTlf = new JLabel(ResourceBundle.getBundle("Etiquetas").getString("BizumGUI.AskTlfLabel"));
        lblTlf.setBounds(50, 70, 131, 30);
        getContentPane().add(lblTlf);

        JTextField txtTlf = new JTextField();
        txtTlf.setBounds(180, 70, 150, 30);
        getContentPane().add(txtTlf);

        // --- ONARTU BOTOIA ---
        JButton btnOnartu = new JButton(ResourceBundle.getBundle("Etiquetas").getString("Accept"));
        btnOnartu.setFocusable(false);
        btnOnartu.setBounds(50, 120, 280, 40);
        btnOnartu.setBackground(new Color(64, 224, 208));
        btnOnartu.setForeground(Color.WHITE);

        btnOnartu.addActionListener(e -> {
            BLFacade facade = MainGUI.getBusinessLogic();
            float saldoBerria = isSarrera ? user.getDirua() + kopurua : user.getDirua() - kopurua;

            // 1. Datu basean saldoa aldatu
            facade.updateSaldo(user.getEmail(), saldoBerria);

            // 2. GAKOA: MainGUI-ko estatikoa berritu (Saioa hasi bezala egiteko)
            MainGUI.saioaIndarrezBerritu();

            JOptionPane.showMessageDialog(this, ResourceBundle.getBundle("Etiquetas").getString("BizumGUI.Done"));

            // 3. Aurreko leihora joan eta freskatu
            this.dispose();
            if (previousFrame != null) {
                previousFrame.setVisible(true);
                if (previousFrame instanceof TransakzioakGUI) {
                    ((TransakzioakGUI) previousFrame).freskatuTaula();
                }
            }
        });

        getContentPane().add(btnOnartu);

        // --- ITXI BOTOIA (Zuk eskatutakoa) ---
        JButton btnItxi = new JButton(ResourceBundle.getBundle("Etiquetas").getString("Close"));
        btnItxi.setFocusable(false);
        btnItxi.setBounds(50, 170, 280, 40);
        btnItxi.setBackground(Color.LIGHT_GRAY);

        btnItxi.addActionListener(e -> {
            this.dispose(); // Leiho hau itxi
            if (previousFrame != null) {
                previousFrame.setVisible(true); // TrantsakzioakGUI erakutsi
            }
        });
        getContentPane().add(btnItxi);

        this.setLocationRelativeTo(null);
    }
}
```

Kode Orain:

```java
public class BizumGUI extends JFrame {

    private float kopurua;
    private final Seller user;
    private JFrame previousFrame;
    private boolean isSarrera;

    public Seller getUser() {
        return user;
    }

    public BizumGUI(float kopurua, Seller user, JFrame previousFrame, boolean isSarrera) {
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        ImageIcon icon = new ImageIcon(getClass().getClassLoader().getResource("images/Logo.png"));
        setIconImage(icon.getImage());
        this.kopurua = kopurua;
        this.user = user;
        this.previousFrame = previousFrame;
        this.isSarrera = isSarrera;

        setTitle(ResourceBundle.getBundle("Etiquetas").getString("BizumGUI.Title"));
        setSize(400, 300);
        getContentPane().setLayout(null);
        this.getContentPane().setBackground(Color.WHITE);

        JLabel lblInfo = new JLabel((isSarrera ? ResourceBundle.getBundle("Etiquetas").getString("BizumGUI.Add")+": " : ResourceBundle.getBundle("Etiquetas").get
        lblInfo.setBounds(50, 20, 300, 30);
        lblInfo.setHorizontalAlignment(SwingConstants.CENTER);
        lblInfo.setFont(new Font("Segoe UI", Font.BOLD, 16));
        getContentPane().add(lblInfo);

        JLabel lblTlf = new JLabel(ResourceBundle.getBundle("Etiquetas").getString("BizumGUI.AskTlfLabel"));
        lblTlf.setBounds(50, 70, 131, 30);
        getContentPane().add(lblTlf);

        JTextField txtTlf = new JTextField();
        txtTlf.setBounds(180, 70, 150, 30);
        getContentPane().add(txtTlf);

        // --- ONARTU BOTOIA ---
        JButton btnOnartu = new JButton(ResourceBundle.getBundle("Etiquetas").getString("Accept"));
        btnOnartu.setFocusable(false);
        btnOnartu.setBounds(50, 120, 280, 40);
        btnOnartu.setBackground(new Color(64, 224, 208));
        btnOnartu.setForeground(Color.WHITE);

        btnOnartu.addActionListener(e -> {
            BLFacade facade = MainGUI.getBusinessLogic();
            float saldoBerria = isSarrera ? user.getDirua() + kopurua : user.getDirua() - kopurua;

            // 1. Datu basean saldoa aldatu
            facade.updateSaldo(user.getEmail(), saldoBerria);

            // 2. GAKOA: MainGUI-ko estatikoa berritu
            MainGUI.saioaIndarrezBerritu();

            JOptionPane.showMessageDialog(this, ResourceBundle.getBundle("Etiquetas").getString("BizumGUI.Done"));

            // 3. Kode duplikatua saihesteko metodo laguntzailea deitu
            itxiEtaItzuli();
        });
        getContentPane().add(btnOnartu);

        // --- ITXI BOTOIA ---
        JButton btnItxi = new JButton(ResourceBundle.getBundle("Etiquetas").getString("Close"));
        btnItxi.setFocusable(false);
        btnItxi.setBounds(50, 170, 280, 40);
        btnItxi.setBackground(Color.LIGHT_GRAY);

        btnItxi.addActionListener(e -> {
            itxiEtaItzuli();
        });
        getContentPane().add(btnItxi);

        this.setLocationRelativeTo(null);
    }

    /**
     * Leihoa itxi eta aurreko pantailara itzultzeko logika berrerabilgarria,
     * "Duplicate code" arazoa ekiditeko sortua.
     */
    private void itxiEtaItzuli() {
        this.dispose();
        if (previousFrame != null) {
            previousFrame.setVisible(true);
            if (previousFrame instanceof TransakzioakGUI) {
                ((TransakzioakGUI) previousFrame).freskatuTaula();
            }
        }
    }
}
```

Aukeratutako klasea: BizumGUI klasea.

Zergatik aukeratu den: SonarLint/SonarCloud tresnek klase honetan **% 17.9ko kode-errepikapena** (*Duplicated Lines*) zegoela adierazi zuten. Zehazki, leihoaren bi botoien (btnOnartu eta btnItxi) barruan, leihoa itxi eta aurreko pantailara itzultzeko logika berbera kopiatuta zegoen bi lekuetan.

Zer egin den: Kode errepikatu hori metodo laguntzaile pribatu eta berrerabilgarri batera ateratzea erabaki da (itxiEtaItzuli()). Orain, bi botoiek metodo berri honi deitzen diote; horrela, kode-errepikapena ezabatu eta garbiago geratu da.

Enara Anitua Goitiandia:

Hasierako Kodea:

```
public class ErregistratuGUI extends JFrame {

    private static final long serialVersionUID = 1L;
    private JPanel contentPane;
    private JTextField textFieldEmail;
    private JTextField textFieldIzena;
    private JPasswordField passwordField;
    private JPasswordField passwordField_1;
    private JLabel lblError;

    public ErregistratuGUI() {
        ImageIcon icon = new ImageIcon(getClass().getClassLoader().getResource("images/Logo.png"));
        setIconImage(icon.getImage());
        setFocusable(false);
        setTitle(ResourceBundle.getBundle("Etiquetas").getString("ErregistratuGUI.MainTitle"));
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        setBounds(100, 100, 480, 360);


        this.addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                MainGUI main = new MainGUI("seller3@gmail.com");
                main.setVisible(true);
            }
        });

        contentPane = new JPanel();
        contentPane.setForeground(Color.DARK_GRAY);
        contentPane.setBackground(Color.WHITE);
        contentPane.setFocusable(false);
        contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));
        setContentPane(contentPane);
        contentPane.setLayout(null);

        // --- ETIKETAK (Labels) ---
        JLabel lblEmail = new JLabel("E-MAIL");
        lblEmail.setFont(new Font("Segoe UI Semibold", Font.PLAIN, 14));
        lblEmail.setForeground(Color.DARK_GRAY);
        lblEmail.setFocusable(false);
        lblEmail.setBounds(49, 47, 91, 14);
        contentPane.add(lblEmail);

        JLabel lblIzena = new JLabel(ResourceBundle.getBundle("Etiquetas").getString("ErregistratuGUI.NameLabel"));
        lblIzena.setFont(new Font("Segoe UI Semibold", Font.PLAIN, 14));
        lblIzena.setForeground(Color.DARK_GRAY);
        lblIzena.setFocusable(false);
        lblIzena.setBounds(49, 96, 120, 18);
        contentPane.add(lblIzena);

        JLabel lblPass = new JLabel(ResourceBundle.getBundle("Etiquetas").getString("PasswordLabel"));
        lblPass.setFont(new Font("Segoe UI Semibold", Font.PLAIN, 14));
        lblPass.setForeground(Color.DARK_GRAY);
        lblPass.setFocusable(false);
        lblPass.setBounds(46, 142, 91, 18);
        contentPane.add(lblPass);

        JLabel lblPassRep = new JLabel(ResourceBundle.getBundle("Etiquetas").getString("ErregistratuGUI.RepeatPasswordLabel"));
        lblPassRep.setFont(new Font("Segoe UI Semibold", Font.PLAIN, 14));
        lblPassRep.setForeground(Color.DARK_GRAY);
        lblPassRep.setFocusable(false);
        lblPassRep.setBounds(46, 195, 150, 18);
        contentPane.add(lblPassRep);
```

```
        // --- TESTU EREMUAK (Fields) ---
        textFieldEmail = new JTextField();
        textFieldEmail.setBorder(new MatteBorder(0, 0, 2, 0, (Color) new Color(64, 224, 208)));
        textFieldEmail.setForeground(Color.DARK_GRAY);
        textFieldEmail.setBackground(Color.WHITE);
        textFieldEmail.setBounds(208, 45, 180, 20);
        contentPane.add(textFieldEmail);

        textFieldIzena = new JTextField();
        textFieldIzena.setBorder(new MatteBorder(0, 0, 2, 0, (Color) new Color(64, 224, 208)));
        textFieldIzena.setForeground(Color.DARK_GRAY);
        textFieldIzena.setBackground(Color.WHITE);
        textFieldIzena.setBounds(208, 94, 180, 20);
        contentPane.add(textFieldIzena);

        passwordField = new JPasswordField();
        passwordField.setBorder(new MatteBorder(0, 0, 2, 0, (Color) new Color(64, 224, 208)));
        passwordField.setForeground(Color.DARK_GRAY);
        passwordField.setBackground(Color.WHITE);
        passwordField.setBounds(208, 140, 180, 20);
        contentPane.add(passwordField);

        passwordField_1 = new JPasswordField();
        passwordField_1.setBorder(new MatteBorder(0, 0, 2, 0, (Color) new Color(64, 224, 208)));
        passwordField_1.setForeground(Color.DARK_GRAY);
        passwordField_1.setBackground(Color.WHITE);
        passwordField_1.setBounds(208, 193, 180, 20);
        contentPane.add(passwordField_1);

        lblError = new JLabel("");
        lblError.setFont(new Font("Segoe UI", Font.ITALIC, 11));
        lblError.setFocusable(false);
        lblError.setHorizontalAlignment(SwingConstants.CENTER);
        lblError.setForeground(Color.RED);
        lblError.setBounds(49, 267, 369, 56);
        contentPane.add(lblError);

        // --- BOTOIA ETA LOGIKA ---
        JButton btnErregistratu = new JButton(ResourceBundle.getBundle("Etiquetas").getString("AukerakIkusiGUI.RegisterButton"));
        btnErregistratu.setBorder(new LineBorder(new Color(64, 224, 208)));
        btnErregistratu.setFont(new Font("Segoe UI", Font.BOLD, 14));
        btnErregistratu.setForeground(new Color(255, 255, 255));
        btnErregistratu.setBackground(new Color(64, 224, 208));
        btnErregistratu.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
        btnErregistratu.setFocusable(false);
        btnErregistratu.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                lblError.setText("");

                String email = textFieldEmail.getText();
                String izena = textFieldIzena.getText();
                String pass = new String(passwordField.getPassword());
                String passRep = new String(passwordField_1.getPassword());

                if (email.isEmpty() || izena.isEmpty() || pass.isEmpty() || passRep.isEmpty()) {
                    lblError.setText(ResourceBundle.getBundle("Etiquetas").getString("ErregistratuGUI.CompleteFieldAlert"));
                    return;
                }

                if (!pasahitzaSeguruaDa(pass)) {
                    lblError.setText("<html><center>"+ResourceBundle.getBundle("Etiquetas").getString("ErregistratuGUI.PasswordError1")+"<br>"
                                + ResourceBundle.getBundle("Etiquetas").getString("ErregistratuGUI.PasswordError2")+"</center></html>");
                    return;
                }

                if (!pass.equals(passRep)) {
                    lblError.setText(ResourceBundle.getBundle("Etiquetas").getString("ErregistratuGUI.DifferentPasswordAlert"));
                    return;
                }

                try {
                    BLFacade facade = MainGUI.getBusinessLogic();

                    if (facade.isBlacklisted(email)) {
                        lblError.setText(ResourceBundle.getBundle("Etiquetas").getString("ErregistratuGUI.BlacklistedErrorAlert"));
                        return;
                    }

                    Seller s = facade.register(email, izena, pass);

                    if (s != null) {
                        JOptionPane.showMessageDialog(null, ResourceBundle.getBundle("Etiquetas").getString("ErregistratuGUI.CorrectAlert"));
                        SaloaMasIGUI saloa = new SaloaMasIGUI();
                        saloa.setVisible(true);
                        dispose();
                    } else {
                        lblError.setText(ResourceBundle.getBundle("Etiquetas").getString("ErregistratuGUI.ExistingMailError"));
                    }
                } catch (Exception ex) {
                    lblError.setText("Error: " + ex.getMessage());
                }
            }
        });
        btnErregistratu.setBounds(123, 235, 205, 30);
        contentPane.add(btnErregistratu);
    }

    private boolean pasahitzaSeguruaDa(String password) {
        String pattern = "^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z])(?=.*[@#$%^&+=!?.])(?=\\S+$).{8,12}$";
        return password.matches(pattern);
    }
```

Kodea orain:

```
public class ErregistratuGUI extends JFrame {

    private static final long serialVersionUID = 1L;
    private JPanel contentPane;
    private JTextField textFieldEmail;
    private JTextField textFieldIzena;
    private JPasswordField passwordField;
    private JPasswordField passwordField_1;
    private JLabel lblError;

    public ErregistratuGUI() {
        ImageIcon icon = new ImageIcon(getClass().getClassLoader().getResource("images/Logo.png"));
        setIconImage(icon.getImage());
        setFocusable(false);
        setTitle(ResourceBundle.getBundle("Etiquetas").getString("ErregistratuGUI.MainTitle"));
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        setBounds(100, 100, 480, 360);

        this.addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                MainGUI main = new MainGUI("seller3@gmail.com");
                main.setVisible(true);
            }
        });

        contentPane = new JPanel();
        contentPane.setForeground(Color.DARK_GRAY);
        contentPane.setBackground(Color.WHITE);
        contentPane.setFocusable(false);
        contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));
        setContentPane(contentPane);
        contentPane.setLayout(null);

        // --- ETIKETAK (Labels) ---
        JLabel lblEmail = new JLabel("E-MAIL");
        lblEmail.setFont(new Font("Segoe UI Semibold", Font.PLAIN, 14));
        lblEmail.setForeground(Color.DARK_GRAY);
        lblEmail.setFocusable(false);
        lblEmail.setBounds(49, 47, 91, 14);
        contentPane.add(lblEmail);

        JLabel lblIzena = new JLabel(ResourceBundle.getBundle("Etiquetas").getString("ErregistratuGUI.NameLabel"));
        lblIzena.setFont(new Font("Segoe UI Semibold", Font.PLAIN, 14));
        lblIzena.setForeground(Color.DARK_GRAY);
        lblIzena.setFocusable(false);
        lblIzena.setBounds(49, 96, 120, 18);
        contentPane.add(lblIzena);

        JLabel lblPass = new JLabel(ResourceBundle.getBundle("Etiquetas").getString("PasswordLabel"));
        lblPass.setFont(new Font("Segoe UI Semibold", Font.PLAIN, 14));
        lblPass.setForeground(Color.DARK_GRAY);
        lblPass.setFocusable(false);
        lblPass.setBounds(46, 142, 91, 18);
        contentPane.add(lblPass);
```

```
JLabel lblPassRep = new JLabel(ResourceBundle.getBundle("Etiquetas").getString("ErregistratuGUI.RepeatPasswordLabel"));
lblPassRep.setFont(new Font("Segoe UI Semibold", Font.PLAIN, 14));
lblPassRep.setForeground(Color.DARK_GRAY);
lblPassRep.setFocusable(false);
lblPassRep.setBounds(46, 195, 150, 18);
contentPane.add(lblPassRep);

// --- TESTU EREMUAK (Metodo laguntzaileak erabiliz kode-bikoizketa ekiditeko) ---
textFieldEmail = textEremuaSortu(45);
contentPane.add(textFieldEmail);

textFieldIzena = textEremuaSortu(94);
contentPane.add(textFieldIzena);

passwordField = passwordEremuaSortu(140);
contentPane.add(passwordField);

passwordField_1 = passwordEremuaSortu(193);
contentPane.add(passwordField_1);

lblError = new JLabel("");
lblError.setFont(new Font("Segoe UI", Font.ITALIC, 11));
lblError.setFocusable(false);
lblError.setHorizontalAlignment(SwingConstants.CENTER);
lblError.setForeground(Color.RED);
lblError.setBounds(49, 267, 369, 56);
contentPane.add(lblError);

// --- BOTOIA ETA LOGIKA ---
JButton btnErregistratu = new JButton(ResourceBundle.getBundle("Etiquetas").getString("AukerakIkusiGUI.RegisterButton"));
btnErregistratu.setBorder(new LineBorder(new Color(64, 224, 208)));
btnErregistratu.setFont(new Font("Segoe UI", Font.BOLD, 14));
btnErregistratu.setForeground(new Color(255, 255, 255));
btnErregistratu.setBackground(new Color(64, 224, 208));
btnErregistratu.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
btnErregistratu.setFocusable(false);
btnErregistratu.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        lblError.setText("");

        String email = textFieldEmail.getText();
        String izena = textFieldIzena.getText();
        String pass = new String(passwordField.getPassword());
        String passRep = new String(passwordField_1.getPassword());

        if (email.isEmpty() || izena.isEmpty() || pass.isEmpty() || passRep.isEmpty()) {
            lblError.setText(ResourceBundle.getBundle("Etiquetas").getString("ErregistratuGUI.CompleteFieldAlert"));
            return;
        }

        if (!pasahitzaSeguruaDa(pass)) {
            lblError.setText("<html><center>"+ResourceBundle.getBundle("Etiquetas").getString("ErregistratuGUI.PasswordError1")+"<br>"
                    + ResourceBundle.getBundle("Etiquetas").getString("ErregistratuGUI.PasswordError2")+"</center></html>");
            return;
        }
```

```
                String email = textFieldEmail.getText();
                String izena = textFieldIzena.getText();
                String pass = new String(passwordField.getPassword());
                String passRep = new String(passwordField_1.getPassword());

                if (email.isEmpty() || izena.isEmpty() || pass.isEmpty() || passRep.isEmpty()) {
                    lblError.setText(ResourceBundle.getBundle("Etiquetas").getString("ErregistratuGUI.CompleteFieldAlert"));
                    return;
                }

                if (!pasahitzaSeguruaDa(pass)) {
                    lblError.setText("<html><center>"+ResourceBundle.getBundle("Etiquetas").getString("ErregistratuGUI.PasswordError1")+"<br>"
                            + ResourceBundle.getBundle("Etiquetas").getString("ErregistratuGUI.PasswordError2")+"</center></html>");
                    return;
                }

                if (!pass.equals(passRep)) {
                    lblError.setText(ResourceBundle.getBundle("Etiquetas").getString("ErregistratuGUI.DifferentPasswordAlert"));
                    return;
                }

                try {
                    BLFacade facade = MainGUI.getBusinessLogic();

                    if (facade.isBlacklisted(email)) {
                        lblError.setText(ResourceBundle.getBundle("Etiquetas").getString("ErregistratuGUI.BlacklistedErrorAlert"));
                        return;
                    }

                    Seller s = facade.register(email, izena, pass);

                    if (s != null) {
                        JOptionPane.showMessageDialog(null, ResourceBundle.getBundle("Etiquetas").getString("ErregistratuGUI.CorrectAlert"));
                        SaioaHasiGUI saioa = new SaioaHasiGUI();
                        saioa.setVisible(true);
                        dispose();
                    } else {
                        lblError.setText(ResourceBundle.getBundle("Etiquetas").getString("ErregistratuGUI.ExistingMailError"));
                    }
                } catch (Exception ex) {
                    lblError.setText("Error: " + ex.getMessage());
                }
            }
        });
        btnErregistratu.setBounds(123, 235, 205, 30);
        contentPane.add(btnErregistratu);
    }

    private boolean pasahitzaSeguruaDa(String password) {
        String pattern = "^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z])(?=.*[@#$%^*&+=!?.])(?=\\S+$).{8,12}$";
        return password.matches(pattern);
    }

    private JTextField textEremuaSortu(int yPos) {
        JTextField tf = new JTextField();
        tf.setBorder(new MatteBorder(0, 0, 2, 0, (Color) new Color(64, 224, 208)));
        tf.setForeground(Color.DARK_GRAY);
        tf.setBackground(Color.WHITE);
        tf.setBounds(208, yPos, 180, 20);
        return tf;
    }

    private JPasswordField passwordEremuaSortu(int yPos) {
        JPasswordField pf = new JPasswordField();
        pf.setBorder(new MatteBorder(0, 0, 2, 0, (Color) new Color(64, 224, 208)));
        pf.setForeground(Color.DARK_GRAY);
        pf.setBackground(Color.WHITE);
        pf.setBounds(208, yPos, 180, 20);
        return pf;
    }
}
```

Aukeratutako klasea: ErregistratuGUI klasea.

Zergatik aukeratu den: Aurreko BizumGUI klasean bezala, SonarCloud-ek adierazi zigun klase honetan ere kode-errepikapena zegoela. Zehazki, testu-eremuak eta pasahitz-eremuak sortzerakoan, kode guzti horretan aldatzen den gauza bakarra kokapen zenbaki bat zen. Baina, estiloa, mugak eta neurriak berdinak ziren denbora guztiak, kode errepikapena sortuz.

Zer egin den: Kode errepikatu horiek metodo laguntzaile pribatu eta berriz erabiltzeko ahalmena duten metodoetara bihurtu ditugu. Parametro bat sartuko zaie metodo bakoitzari, posizioa adieraziko duena (izan ere esan dut kodean aldatzen den gauza bakarra posizioa da).

Orain, eremuak sortzerakoa metodo honei egiten zaie deia, horrela kode errepikapena ezabatuz, klasea askoz sinpleagoa eta garbiago utziz.

Hodei Salegi Aranguena

Hasierako Kodea:

```
package gui;

import javax.swing.*;

public class AukerakErosiGUI extends JFrame {

    // Saioa gordetzeko aldagai estatikoa
    public static Seller unekoErabiltzailea = null;

    private static final long serialVersionUID = 1L;
    private JPanel jContentPane = null;

    private JButton btnErregistratu = null;
    private JButton btnSaioaHasi = null;
    private JButton btnBisitari = null;

    private static BLFacade appFacadeInterface;
    private static final String ETIQUETAS="Etiquetas";

    public static BLFacade getBusinessLogic(){
        return appFacadeInterface;
    }

    public static void setBussinessLogic (BLFacade facade){
        appFacadeInterface=facade;
    }

    protected JLabel jLabelSelectOption;
    private JPanel panelHizkuntzak;
    private final ButtonGroup buttonGroup = new ButtonGroup();

    public AukerakErosiGUI() {
        super();
        ImageIcon icon = new ImageIcon(getClass().getClassLoader().getResource("images/Logo.png"));
        setIconImage(icon.getImage());
        this.setSize(495, 420); // Tamaina pixka bat handitu dugu erosotasunagatik
        this.getContentPane().setLayout(null);
        this.getContentPane().setBackground(Color.WHITE);

        jLabelSelectOption = new JLabel(ResourceBundle.getBundle(ETIQUETAS).getString("MainGUI.SelectOption"));
        jLabelSelectOption.setFont(new Font("Tahoma", Font.BOLD, 20));
        jLabelSelectOption.setForeground(Color.DARK_GRAY);
        jLabelSelectOption.setHorizontalAlignment(SwingConstants.CENTER);
        jLabelSelectOption.setBounds(0, 20, 481, 40);
        this.getContentPane().add(jLabelSelectOption);

        // 1. ERREGISTRATU BOTOIA
        btnErregistratu = new JButton(ResourceBundle.getBundle(ETIQUETAS).getString("AukerakIkusiGUI.RegisterButton")); //$NON-NLS-1$ //$NON-NL
        btnErregistratu.setFont(new Font("Segoe UI", Font.BOLD, 14));
        btnErregistratu.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
        btnErregistratu.setForeground(Color.WHITE);
        btnErregistratu.setBackground(new Color(64, 224, 208));
        btnErregistratu.setBounds(50, 80, 380, 50);
        btnErregistratu.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                // Leiho berria ireki eta hau ezkutatu (aukerakoa)
                JFrame a = new ErregistratuGUI();
                a.setVisible(true);
            }
        });
        this.getContentPane().add(btnErregistratu);

        // 2. SAIOA HASI BOTOIA
        btnSaioaHasi = new JButton(ResourceBundle.getBundle(ETIQUETAS).getString("AukerakIkusiGUI.LoginButton")); //$NON-NLS-1$ //$NON-NLS-2$
        btnSaioaHasi.setFont(new Font("Segoe UI", Font.BOLD, 14));
        btnSaioaHasi.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
        btnSaioaHasi.setForeground(new Color(64, 224, 208));
        btnSaioaHasi.setBackground(Color.WHITE);
        btnSaioaHasi.setBorder(new LineBorder(new Color(64, 224, 208)));
        btnSaioaHasi.setBounds(50, 145, 380, 50);
        btnSaioaHasi.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                // Leiho berria ireki
                JFrame a = new SaioaHasiGUI();
                a.setVisible(true);
            }
        });
        this.getContentPane().add(btnSaioaHasi);

        // 3. BISITARI MODUAN KONTSULTATU (Saioa hasi gabe)
        btnBisitari = new JButton(ResourceBundle.getBundle(ETIQUETAS).getString("AukerakIkusiGUI.ContinueButton")); //$NON-NLS-1$ //$NON-NLS-2$
        btnBisitari.setFont(new Font("Segoe UI", Font.PLAIN, 13));
        btnBisitari.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
        btnBisitari.setForeground(Color.GRAY);
        btnBisitari.setBackground(new Color(245, 245, 245));
        btnBisitari.setBounds(50, 210, 380, 40);
```

```
        btnBisitari.addActionListener(e -> {
            unekoErabiltzailea = null;
            JFrame a = new QuerySalesGUI(null);
            a.setVisible(true);
        });
        this.getContentPane().add(btnBisitari);

        // 4. HIZKUNTZAK
        panelHizkuntzak = new JPanel();
        panelHizkuntzak.setBackground(Color.WHITE);
        panelHizkuntzak.setBounds(0, 280, 481, 50);
        this.getContentPane().add(panelHizkuntzak);

        JRadioButton rdbtnEus = new JRadioButton("Euskara");
        rdbtnEus.setSelected(true);
        rdbtnEus.addActionListener(e -> {
            Locale.setDefault(new Locale("eu"));
            paintAgain();
        });
        buttonGroup.add(rdbtnEus);
        panelHizkuntzak.add(rdbtnEus);

        JRadioButton rdbtnEs = new JRadioButton("Castellano");
        rdbtnEs.addActionListener(e -> {
            Locale.setDefault(new Locale("es"));
            paintAgain();
        });
        buttonGroup.add(rdbtnEs);
        panelHizkuntzak.add(rdbtnEs);

        JRadioButton rdbtnEn = new JRadioButton("English");
        rdbtnEn.addActionListener(e -> {
            Locale.setDefault(new Locale("en"));
            paintAgain();
        });
        buttonGroup.add(rdbtnEn);
        panelHizkuntzak.add(rdbtnEn);

        this.setTitle(ResourceBundle.getBundle(ETIQUETAS).getString("MainGUI.MainTitle") + ": "+ResourceBundle.getBundle(ETIQUETAS).getString("Welcome"));

        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                System.exit(1);
            }
        });

        this.setLocationRelativeTo(null);
    }

    private void paintAgain() {
        jLabelSelectOption.setText(ResourceBundle.getBundle(ETIQUETAS).getString("MainGUI.SelectOption"));
        this.setTitle(ResourceBundle.getBundle(ETIQUETAS).getString("MainGUI.MainTitle") + ": "+ResourceBundle.getBundle(ETIQUETAS).getString("Welcome"));
    }

    public static void saioaIndarrezBerritu() {
        if (unekoErabiltzailea != null) {
            unekoErabiltzailea = getBusinessLogic().getUser(unekoErabiltzailea.getEmail());
        }
    }
}
```

Kodea orain:

```java
package gui;

import javax.swing.*;

public class AukerakErosiGUI extends JFrame {

    // Saioa sortzeko aldagai estatikoa
    public static Seller unekoErabiltzailea = null;

    private static final long serialVersionUID = 1L;
    private JPanel jContentPane = null;

    private JButton btnErregistratu = null;
    private JButton btnSaioaHasi = null;
    private JButton btnBisitari = null;

    private static BLFacade appFacadeInterface;
    private static final String ETIQUETAS="Etiquetas";

    public static BLFacade getBusinessLogic(){
        return appFacadeInterface;
    }

    public static void setBussinessLogic (BLFacade facade){
        appFacadeInterface=facade;
    }

    protected JLabel jLabelSelectOption;
    private JPanel panelHizkuntzak;
    private final ButtonGroup buttonGroup = new ButtonGroup();

    public AukerakErosiGUI() {
        super();
        ImageIcon icon = new ImageIcon(getClass().getClassLoader().getResource("images/Logo.png"));
        setIconImage(icon.getImage());
        this.setSize(495, 420); // Tamaina pixka bat handitu dugu erosotasunagatik
        this.getContentPane().setLayout(null);
        this.getContentPane().setBackground(Color.WHITE);

        jLabelSelectOption = new JLabel(ResourceBundle.getBundle(ETIQUETAS).getString("MainGUI.SelectOption"));
        jLabelSelectOption.setFont(new Font("Tahoma", Font.BOLD, 20));
        jLabelSelectOption.setForeground(Color.DARK_GRAY);
        jLabelSelectOption.setHorizontalAlignment(SwingConstants.CENTER);
        jLabelSelectOption.setBounds(0, 20, 481, 40);
        this.getContentPane().add(jLabelSelectOption);

        // 1. ERREGISTRATU BOTOIA
        btnErregistratu = new JButton(ResourceBundle.getBundle(ETIQUETAS).getString("AukerakIkusiGUI.RegisterButton")); //$NON-NLS-1$ //$NON-NL
        btnErregistratu.setFont(new Font("Segoe UI", Font.BOLD, 14));
        btnErregistratu.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
        btnErregistratu.setForeground(Color.WHITE);
        btnErregistratu.setBackground(new Color(64, 224, 208));
        btnErregistratu.setBounds(50, 80, 380, 50);
        btnErregistratu.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                // Leiho berria ireki eta hau ezkutatu (aukerakoa)
                JFrame a = new ErregistratuGUI();
                a.setVisible(true);
            }
        });
        this.getContentPane().add(btnErregistratu);

        // 2. SAIOA HASI BOTOIA
        btnSaioaHasi = new JButton(ResourceBundle.getBundle(ETIQUETAS).getString("AukerakIkusiGUI.LoginButton")); //$NON-NLS-1$ //$NON-NLS-2$
        btnSaioaHasi.setFont(new Font("Segoe UI", Font.BOLD, 14));
        btnSaioaHasi.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
        btnSaioaHasi.setForeground(new Color(64, 224, 208));
        btnSaioaHasi.setBackground(Color.WHITE);
        btnSaioaHasi.setBorder(new LineBorder(new Color(64, 224, 208)));
        btnSaioaHasi.setBounds(50, 145, 380, 50);
        btnSaioaHasi.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                // Leiho berria ireki
                JFrame a = new SaioaHasiGUI();
                a.setVisible(true);
            }
        });
        this.getContentPane().add(btnSaioaHasi);

        // 3. BISITARI MODUAN KONTSULTATU (Saioa hasi gabe)
        btnBisitari = new JButton(ResourceBundle.getBundle(ETIQUETAS).getString("AukerakIkusiGUI.ContinueButton")); //$NON-NLS-1$ //$NON-NLS-2$
        btnBisitari.setFont(new Font("Segoe UI", Font.PLAIN, 13));
        btnBisitari.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
        btnBisitari.setForeground(Color.GRAY);
        btnBisitari.setBackground(new Color(245, 245, 245));
        btnBisitari.setBounds(50, 210, 380, 40);
```

```
        btnBisitari.addActionListener(e -> {
            unekoErabiltzailea = null;
            JFrame a = new QuerySalesGUI(null);
            a.setVisible(true);
        });
        this.getContentPane().add(btnBisitari);

        // 4. HIZKUNTZAK
        panelHizkuntzak = new JPanel();
        panelHizkuntzak.setBackground(Color.WHITE);
        panelHizkuntzak.setBounds(0, 280, 481, 50);
        this.getContentPane().add(panelHizkuntzak);

        JRadioButton rdbtnEus = new JRadioButton("Euskara");
        rdbtnEus.setSelected(true);
        rdbtnEus.addActionListener(e -> {
            String hizkuntza = "eu";
            aldatuHizkuntza(hizkuntza);
        });
        buttonGroup.add(rdbtnEus);
        panelHizkuntzak.add(rdbtnEus);

        JRadioButton rdbtnEs = new JRadioButton("Castellano");
        rdbtnEs.addActionListener(e -> {
            String hizkuntza = "es";
            aldatuHizkuntza(hizkuntza);
        });
        buttonGroup.add(rdbtnEs);
        panelHizkuntzak.add(rdbtnEs);

        JRadioButton rdbtnEn = new JRadioButton("English");
        rdbtnEn.addActionListener(e -> {
            String hizkuntza = "en";
            aldatuHizkuntza(hizkuntza);
        });
        buttonGroup.add(rdbtnEn);
        panelHizkuntzak.add(rdbtnEn);

        this.setTitle(ResourceBundle.getBundle(ETIQUETAS).getString("MainGUI.MainTitle") + ": "+ResourceBundle.getBundle(ETIQUETAS).getString("Welcome"));

        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                System.exit(1);
            }
        });

        this.setLocationRelativeTo(null);
    }

    private void aldatuHizkuntza(String hizkuntza) {
        Locale.setDefault(new Locale(hizkuntza));
        paintAgain();
    }

    private void paintAgain() {
        jLabelSelectOption.setText(ResourceBundle.getBundle(ETIQUETAS).getString("MainGUI.SelectOption"));
        this.setTitle(ResourceBundle.getBundle(ETIQUETAS).getString("MainGUI.MainTitle") + ": "+ResourceBundle.getBundle(ETIQUETAS).getString("Welcome"));
    }

    public static void saioaIndarrezBerritu() {
        if (unekoErabiltzailea != null) {
            unekoErabiltzailea = getBusinessLogic().getUser(unekoErabiltzailea.getEmail());
        }
    }
}
```

Aukeratutako klasea: AukerakErosiGUI klasea.

Zergatik aukeratu den: SonarLint / SonarCloud tresnek adierazi bezala, AukerakErosiGUI klasean kode-errepikapena (Duplicated Lines) zegoen. Zehazki, aplikazioaren hizkuntza aldatzeko hiru JRadioButton botoien entzuleren barruan (addActionListener), aplikazioaren Locale berria ezarri eta interfazea berriro margotzeko funtzionalitate-logika berbera (Locale.setDefault(...) eta paintAgain()) hiru aldiz errepikatuta agertzen zen. "Building Maintainable Software" liburuko 4. kapituluak zera agintzen du: *"Duplicate code / Do not copy code"* (ez kopiatu kodea), kode errepikatuak mantentze-lanak zailtzen dituelako eta etorkizuneko aldaketetan akatsak sortzeko arriskua handitzen duelako.

Zer egin den: Eclipse-ren Refactor -> Extract Method tresnaz baliatuz, bikoiztutako logika hori metodo laguntzaile pribatu eta berrerabilgarri batera atera da (aldatuHizkuntza(String hizkuntzaKodea)). Metodo berri honek hizkuntza-kodea jasotzen du parametro gisa. Orain, hiru botoien entzuleek metodo orokor honi deitzen diote lerro bakar batean; horrela, kode-bikoizketa erabat ezabatu da eta etorkizunean hizkuntza-aldaketaren logika aldatu nahi bada, leku bakar batean aldatzea nahikoa izango da.

5. “KEEP UNIT INTERFACES SMALL” (5.Kapitulua)

Jon Elortegi Mendizabal

Kode lehen:

```
public Sale(String title, String description, int status, float price, Date pubDate, File file, Seller seller) {
    super();

    this.title = title;
    this.description = description;
    this.status = status;
    this.price=price;
    this.pubDate=pubDate;
    this.available = true; // Hasieran salgai dago

    if (file!=null) {
        this.fileName=file.getName();
        try {
            BufferedImage img1 = ImageIO.read(file);
            String path="src/main/resources/images/";
            File outputfile = new File(path+file.getName());
            ImageIO.write(img1, "png", outputfile);
        } catch(IOException ex) {
            // Errore kudeaketa
        }
    }

    this.seller = seller;
}
```

Kodea Orain:

```
/**
 * ERREFAKTORIZATUTA: "Keep unit interfaces small" jarraibidea jarraituz,
 * 7 parametro banaka pasa beharrean SaleData objektu bakarra jasotzen du.
 */
public Sale(SaleData data) {
    super();

    this.title = data.getTitle();
    this.description = data.getDescription();
    this.status = data.getStatus();
    this.price = data.getPrice();
    this.pubDate = data.getPubDate();
    this.available = true; // Hasieran salgai dago

    File file = data.getFile();
    if (file != null) {
        this.fileName = file.getName();
        try {
            BufferedImage img1 = ImageIO.read(file);
            String path = "src/main/resources/images/";
            File outputfile = new File(path + file.getName());
            ImageIO.write(img1, "png", outputfile);
        } catch(IOException ex) {
            // Errore kudeaketa
        }
    }

    this.seller = data.getSeller();
}
```

SaleData klase berria:

```java
package domain;

import java.io.File;
import java.util.Date;

public class SaleData {
    private String title;
    private String description;
    private int status;
    private float price;
    private Date pubDate;
    private File file;
    private Seller seller;

    public SaleData(String title, String description, int status, float price, Date pubDate, File file, Seller seller) {
        this.title = title;
        this.description = description;
        this.status = status;
        this.price = price;
        this.pubDate = pubDate;
        this.file = file;
        this.seller = seller;
    }

    public String getTitle() { return title; }
    public String getDescription() { return description; }
    public int getStatus() { return status; }
    public float getPrice() { return price; }
    public Date getPubDate() { return pubDate; }
    public File getFile() { return file; }
    public Seller getSeller() { return seller; }
}
```

Aukeratutako kodea: Sale klasearen konstruktorea (public Sale(String title, String description, int status, float price, Date pubDate, File file, Seller seller)).

Zergatik aukeratu den: Unitate bakoitzeko (metodo edo konstruktore) parametro kopurua gehienez 4ra mugatu behar da. Jatorrizko konstruktoreak 7 parametro zituen banaka, eta horrek interfazea handiegia, ulergaitzagoa eta mantentzeko zailagoa bihurtzen zuen.

Zer egin dut:

- SaleData izeneko klase laguntzaile berri bat sortu da, parametro guztiak (title, description, status, price, pubDate, file, seller) bere barnean biltzeko.
- Sale klasearen konstruktorea aldatu da, orain 7 parametro banaka jaso beharrean objektu bakar bat besterik ez baitu jasotzen (SaleData data), horrela interfazea nabarmen txikituz eta garbiago utziz.
- Domain paketeko Seller klasean eta createReclamation-en test-etan ere aldaketa egin behar izan ditut, zehazki “new Sale” egiten zuten aginduetan parametroak aldatu ditut

Enara Anitua Goitiandia:

Hasierako kodea:

```
public Sale createOffer(String title, float price, String description, Seller seller, Demand demand) {
    db.getTransaction().begin();
    try {
        Demand d = db.find(Demand.class, demand.getDemandNumber());
        Seller s = db.find(Seller.class, seller.getEmail());

        Sale offer = new Sale();
        offer.setTitle(title);
        offer.setPrice(price);
        offer.setDescription(description);
        offer.setSeller(s);
        offer.setPublicationDate(new java.util.Date());
        offer.setParentDemand(d);

        db.persist(offer);


        s.addSale(offer);

        db.getTransaction().commit();
        return offer;

    } catch (Exception e) {
        if (db.getTransaction().isActive()) db.getTransaction().rollback();
        e.printStackTrace();
        return null;
    }
}
```

Kodea Orain:

```
public Sale createOffer(CreateOfferParams params) {
    db.getTransaction().begin();
    try {
        Demand d = db.find(Demand.class, params.getDemand().getDemandNumber());
        Seller s = db.find(Seller.class, params.getSeller().getEmail());

        Sale offer = new Sale();
        offer.setTitle(params.getTitle());
        offer.setPrice(params.getPrice());
        offer.setDescription(params.getDescription());
        offer.setSeller(s);
        offer.setPublicationDate(new java.util.Date());
        offer.setParentDemand(d);

        db.persist(offer);
        s.addSale(offer);

        db.getTransaction().commit();
        return offer;

    } catch (Exception e) {
        if (db.getTransaction().isActive()) db.getTransaction().rollback();
        e.printStackTrace();
        return null;
    }
}
```

CreateOfferParams klase berria:

```java
package domain;

import java.io.Serializable;

public class CreateOfferParams implements Serializable {
    private static final long serialVersionUID = 1L;

    private String title;
    private float price;
    private String description;
    private Seller seller;
    private Demand demand;

    public CreateOfferParams(String title, float price, String description, Seller seller, Demand demand) {
        this.title = title;
        this.price = price;
        this.description = description;
        this.seller = seller;
        this.demand = demand;
    }

    // Getter-ak
    public String getTitle() { return title; }
    public float getPrice() { return price; }
    public String getDescription() { return description; }
    public Seller getSeller() { return seller; }
    public Demand getDemand() { return demand; }
}
```

Aukeratutako kodea: DataAccess klaseko createOffer metodoa (public Sale createOffer(String title, float price, String description, Seller seller, Demand demand).

Zergatik aukeratu den: Metodora sartzen den parametro kopurua 4ra mugatu behar delako “Keep unit interfaces small” irizpidearen arabera. CreateOffer-ek hasiera batean 5 parametro zituen hau da irizpidea ez zuen betetzen. Honek, interfazea handiegia, ulertzeko eta mantentzeko zailagoa egiten du, beraz, hori konpondu beharra zegoen.

Zer egin dut:

- CreateOfferParams izeneko klase laguntzaile bat sortu dut, domain paketearen barruan. Hor createOffer metodoaren parametro guztiak (title, price, description, seller, demand) batu ditut. Hasieratzeaz aparte, parametro bakoitzari dagokion getter-a sortu dut.
- Hurrengo pausoa, DataAccess klaseko createOffer metodoa aldatzea izan da. Metodoak 5 parametroak banaka jaso beharrean objektu bakarra jasotzen du (CreateOfferParams params), interfazea txikituz (zehazki irizpideak eskatzen duena).

Hodei Salegi Aranguena

Hasierako kodea:

```java
    // 8 parametro dituen createSale metodoa (lehen createSale2 izena zuena)
    public Sale createSale(String title, String description, int status, float price, Date pubDate, String sellerEmail, File file, Demand parentDemand)
            throws FileNotUploadedException, MustBeLaterThanTodayException, SaleAlreadyExistException {

        if (title == null || pubDate == null || sellerEmail == null) return null;

        try {
            if(pubDate.before(UtilDate.trim(new Date()))) {
                throw new MustBeLaterThanTodayException(ResourceBundle.getBundle(ETIQUETAS).getString("DataAccess.ErrorSaleMustBeLaterThanToday"));
            }
            if (file == null)
                throw new FileNotUploadedException(ResourceBundle.getBundle(ETIQUETAS).getString("DataAccess.ErrorFileNotUploadedException"));

            db.getTransaction().begin();
            Seller seller = db.find(Seller.class, sellerEmail);

            if (seller.doesSaleExist(title)) {
                db.getTransaction().commit();
                throw new SaleAlreadyExistException(ResourceBundle.getBundle(ETIQUETAS).getString("DataAccess.SaleAlreadyExist"));
            }

            // Salmenta sortu
            Sale sale = seller.addSale(title, description, status, price, pubDate, file);

            // Eskaera (parent) badago, lotu
            if (parentDemand != null) {
                Demand d = db.find(Demand.class, parentDemand.getDemandNumber());
                sale.setParentDemand(d);
            }

            db.persist(seller);
            db.getTransaction().commit();
            return sale;

        } catch (Exception e) {
            if (db.getTransaction().isActive()) db.getTransaction().rollback();
            throw e;
        }
    }

// 7 parametrodun jatorrizko metodoa (Demand gabe, null pasatuko dio 8koari)
    public Sale createSale(String title, String description, int status, float price, Date pubDate, String sellerEmail, File file)
            throws FileNotUploadedException, MustBeLaterThanTodayException, SaleAlreadyExistException {
        return createSale(title, description, status, price, pubDate, sellerEmail, file, null);
    }
```

Kodea Orain:

```java
    // 8 parametro dituen createSale metodoa (lehen createSale2 izena zuena)
    public Sale createSale(CreateSaleParams params)
            throws FileNotUploadedException, MustBeLaterThanTodayException, SaleAlreadyExistException {

        if (params.getTitle() == null || params.getDate() == null || params.getSellerEmail() == null) return null;

        try {
            if(params.getDate().before(UtilDate.trim(new Date()))) {
                throw new MustBeLaterThanTodayException(ResourceBundle.getBundle(ETIQUETAS).getString("DataAccess.ErrorSaleMustBeLaterThanToday"));
            }
            if (params.getImage() == null)
                throw new FileNotUploadedException(ResourceBundle.getBundle(ETIQUETAS).getString("DataAccess.ErrorFileNotUploadedException"));

            db.getTransaction().begin();
            Seller seller = db.find(Seller.class, params.getSellerEmail());

            if (seller.doesSaleExist(params.getTitle())) {
                db.getTransaction().commit();
                throw new SaleAlreadyExistException(ResourceBundle.getBundle(ETIQUETAS).getString("DataAccess.SaleAlreadyExist"));
            }

            // Salmenta sortu
            Sale sale = seller.addSale(params.getTitle(), params.getDescription(), params.getNumStatus(), params.getPrice(), params.getDate(), params.getImage());

            // Eskaera (parent) badago, lotu
            if (params.getParent() != null) {
                Demand d = db.find(Demand.class, params.getParent().getDemandNumber());
                sale.setParentDemand(d);
            }

            db.persist(seller);
            db.getTransaction().commit();
            return sale;

        } catch (Exception e) {
            if (db.getTransaction().isActive()) db.getTransaction().rollback();
            throw e;
        }
    }


    // 7 parametrodun metodo laguntzailea
    public Sale createSale(String title, String description, int status, float price, Date pubDate, String sellerEmail, File file)
            throws FileNotUploadedException, MustBeLaterThanTodayException, SaleAlreadyExistException {

        CreateSaleParams params = new CreateSaleParams(title, description, status, price, pubDate, sellerEmail, file, null);
        return createSale(params);
    }
```

CreateSalePArams klase berria:



```java
package domain;

import java.io.File;
import java.io.Serializable;
import java.util.Date;

public class CreateSaleParams implements Serializable {
    private static final long serialVersionUID = 1L;

    private String title;
    private String description;
    private int numStatus;
    private float price;
    private Date date;
    private String sellerEmail;
    private File image;
    private Demand parent;

    public CreateSaleParams(String title, String description, int numStatus, float price, Date date, String sellerEmail, File image, Demand parent) {
        this.title = title;
        this.description = description;
        this.numStatus = numStatus;
        this.price = price;
        this.date = date;
        this.sellerEmail = sellerEmail;
        this.image = image;
        this.parent = parent;
    }

    public String getTitle() { return title; }
    public String getDescription() { return description; }
    public int getNumStatus() { return numStatus; }
    public float getPrice() { return price; }
    public Date getDate() { return date; }
    public String getSellerEmail() { return sellerEmail; }
    public File getImage() { return image; }
    public Demand getParent() { return parent; }
}
```

Aukeratutako kodea: DataAccess klaseko createSale metodoa (public Sale createSale(String title, String description, int status, float price, Date pubDate, String sellerEmail, File file, Demand parentDemand)).

Zergatik aukeratu den: Metodora sartzen den parametro kopurua gehienez 4ra mugatu behar delako "Keep unit interfaces small" irizpidearen arabera. Jatorrizko createSale metodo nagusiak 8 parametro zituen banaka (title, description, status, price, pubDate, sellerEmail, file, parentDemand), hau da, ez zuen liburuko irizpidea betetzen (8 > 4). Honek interfazea handiegia, ulergaitzagoa eta mantentzeko zein probak egiteko zailagoa egiten du.

Zer egin dut:

- CreateSaleParams izeneko datu-klase laguntzailea sortu dut domain paketearen barruan, 8 parametro guztiak bere barnean biltzeko eta bakoitzari dagokion getter-a definituz.
- DataAccess klaseko createSale metodo nagusia errefaktorizatu dut: orain 8 parametroak banaka jaso beharrean, CreateSaleParams objektu bakarra har dezan. Horrela, interfazearen parametro kopurua 8tik 1era murriztu da, liburuaren irizpidea % 100ean betez eta kodearen garbitasuna nahiz erabilgarritasuna nabarmen hobetuz.
- BLFacadeImplementation klaseko createSale metodoa ere egokitu da. Lehen, geruza honek 8 parametroak banan-banan bidaltzen dizkion DataAccess-i; orain, berriz, sarrerako datuekin CreateSaleParams objektua sortzen du eta objektu hori pasatzen dio DataAccess.createSale(params) metodo berriari. Aldaketa honek arkitektura osoan zehar interfazearen koherentzia eta konpilazio zuzena bermatzen ditu.

6. ONDORIOAK

Proiektu honetan zehar, aplikazioaren kodearen kalitatea eta mantentze-erraztasuna nabarmen hobetu dira *"Building Maintainable Software"* liburuko gomendio nagusiei zein **SonarCloud / SonarLint** azterketa-tresnen emaitzei jarraituz. Lehenik eta behin, metodoen luzera gehienez 15 lerrora mugatu da *Extract Method* errefaktorizazio-teknikaren bidez,

unitate luze eta konplexuak azpi-metodo zehatzetan banatuz (hala nola createReclamation, removeSale edo createOffer metodoetan). Horrez gain, unitate bakoitzeko adarkatze-puntuak (baldintzak, zikloak eta try-catch blokeak) gehienez 4ra murriztu dira, balidazioak zein logika konplexuak azpi-metodo independenteetan isolatuz (NullDa, desgaituSalmentak, ezabatuEskaerak). Bestalde, interfaze grafikoetan (BizumGUI, ErregistratuGUI, AukerakErosiGUI) errepikatzen ziren kode-zatiak eta osagaien konfigurazioak metodo pribatu berrerabilgarrietan bideratu dira (itxiEtaItzuli(), textEremuaSortu(), aldatuHizkuntza()), kode bikoiztua erabat ezabatuz. Azkenik, metodo eta konstruktoreetako parametro kopurua 4 baino gutxiagora murrizteko, datu-klase espezifiko berriak sortu dira (SaleData, CreateOfferParams, CreateSaleParams), interfazeak askoz garbiago eta txikiago utziz. Azken batean, egindako errefaktorizazio-lan guzti hauei esker, jatorrizko funtzionalitatea eta portaera erabat babestuz, kode irakurgarriagoa, aztergarriagoa, testagarriagoa eta etorkizuneko mantentze-lanetarako prestatuagoa lortu da.